

Monitoring

역할: FARM/LAB 서버, NFS, GPU와 사용자 container의 실행 상태를 반복 관측하고, Prometheus·Grafana·Alertmanager를 통해 시각화와 경보를 제공한다.

이 문서는 monitoring 문서의 시작점이다. **설계**는 수집 데이터가 어떤 component를 거쳐 metric과 alert가 되는지 설명하고, **운영**은 배포·점검·장애 대응 명령과 코드 위치를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위, 핵심 component와 운영 원칙
설계	exporter, Prometheus, Grafana, Alertmanager, GSS health와 forensics의 데이터 흐름
운영	배포, endpoint 점검, metric/alert 코드 링크, 장애별 진단 순서

한눈에 보는 구조



핵심 component

component	endpoint/산출물	역할
gpu-user-exporter	:30072/metrics, :30072/-/healthy	GPU process를 DB의 실제 사용자/container에 귀속
cluster-monitor-exporter	:30074/metrics, :30074/healthz	mount, GSS, GPU, Docker, container, 연결성, D-state 수집
public health listener	서버별 N89/healthz	외부에서 NAT와 host 도달 가능성 확인
NFS GSS worker	/run · /var/lib state	readiness, canary와 guarded missing-mount recovery
NFS forensics	status JSON, local incident snapshot	제한된 packet/kernel trace를 보존
keytab health check	profile status JSON	AD KVNO, storage keytab과 GSS service drift를 읽기 전용 확인
Prometheus/Alertmanager	cluster별 Kubernetes release	scrape, rule 평가, alert routing
Grafana	FARM NodePort 30080	FARM/LAB datasource와 dashboard를 한 UI에서 제공

운영 경계

- 지속 관측은 **monitoring**, 부팅 orchestration은 **remote-operations**, host desired state는 **server-state**가 소유한다.
- cluster-monitor-exporter**가 직접 mount하지 않는다. 별도 systemd recovery worker가 fstab, DNS/RPC, Kerberos readiness, canary와 D-state gate를 통과한 **missing mount**만 복구한다.
- keytab checker는 AD password, NAS keytab, service와 mount를 변경하지 않는다.
- forensics는 증거를 수집할 뿐 mount/restart/reboot를 수행하지 않는다.
- 자동 복구는 stopped container 시작, SSH 시작, 안전성이 확인된 NVML symlink repair처럼 영향 범위가 제한된 작업에만 사용한다.
- FARM과 LAB Prometheus는 장애와 저장소를 분리하고, Grafana dashboard만 공통 UI에서 datasource UID와 cluster label로 구분한다.

주요 코드

- cluster-monitor-exporter
- gpu-user-exporter
- Prometheus/Alertmanager 설정
- Grafana dashboards
- monitoring Ansible

- [NFS forensics](#)
- [Kerberos/NFS keytab checker](#)

Kerberos credential과 FQDN mount 원칙은 [Kerberos/NFS 설계](#), KVNO·timer·mount 장애 절차는 [Kerberos/NFS 운영](#)을 함께 참고한다.

Monitoring 설계

이 문서는 host의 원시 상태가 metric, alert와 dashboard가 되기까지의 흐름과 자동 복구의 안전 경계를 설명한다.

1. 설계 목표

- FARM/LAB host, GPU, container와 storage 상태를 동일한 metric 체계로 관측한다.
- HTTP process가 살아 있는지와 실제 collection이 갱신되는지를 구분한다.
- NFS **hard** mount와 D-state 상황에서도 monitoring loop 전체가 멈추지 않게 한다.
- alert에는 원인 진단에 필요한 stage와 label을 포함하되 cardinality를 제한한다.
- 관측, 증거 수집과 상태 변경을 분리하여 monitoring이 장애 반경을 넓히지 않는다.
- FARM/LAB Prometheus failure domain은 분리하고 공통 Grafana에서 함께 조회한다.

2. 전체 데이터와 알림 흐름

flowchart TB

subgraph H["FARM/LAB host"]

SRC["/proc · mountinfo · systemd
nvidia-smi · Docker · journal"]

GPU["gpu-user-exporter
:30072"]

CM["cluster-monitor-exporter
:30074 / public :N89"]

GSS["GSS readiness · canary
mount recovery worker"]

FOR["NFS trace ring · watcher
snapshot worker"]

KEY["keytab profile checker
user systemd timer"]

SRC --> GPU

SRC --> CM

GSS -->|"state file"| CM

FOR -->|"status JSON"| CM

end

DB["UID DB"] -->|"container owner cache"| GPU

AD["AD / storage"] -. ->|"read-only KVNO/keytab check"| KEY

GPU -->|"Prometheus metrics"| PF["FARM Prometheus"]

CM -->|"Prometheus metrics"| PF

GPU -->|"Prometheus metrics"| PL["LAB Prometheus"]

CM -->|"Prometheus metrics"| PL

PF --> GRAF["FARM Grafana"]

PL -->|"prometheus-lab datasource"| GRAF

PF --> AM["Alertmanager"]

PL --> AM

AM --> RELAY["localhost notify relay"]

RELAY --> API["internal Slack notify API"]

EXT["Google Apps Script"] <-->|"public health / heartbeat"| CM

데이터 plane과 control plane

종류	예	설계 원칙
관측 data plane	/proc, nvidia-smi, Docker inspect, mountinfo, state JSON	bounded timeout, read-only 우선
metric plane	exporter /metrics → Prometheus	scrape 실패와 collection stale을 별도 표시
alert plane	Prometheus rule → Alertmanager → relay	secret은 Kubernetes Secret, alert label로 credential 전달 금지
visualization plane	FARM/LAB datasource → Grafana	datasource UID와 cluster label로 환경 구분

종류	예	설계 원칙
제한된 control plane	container start, SSH start, NVML repair, missing mount worker	명시적 enable, 사전조건, retry/inhibit 필요

3. cluster-monitor-exporter

Background collection과 freshness

exporter는 HTTP 요청 때마다 무거운 host 명령을 실행하지 않고 background loop의 마지막 결과를 text format으로 제공한다. `/healthz` 는 process 응답뿐 아니라 마지막 성공 collection이 `metrics_stale_after` 이내인지 확인한다.

이 구분이 필요한 이유는 HTTP server는 살아 있지만 child command나 collection goroutine이 멈춘 상태를 `up=1` 만으로는 찾을 수 없기 때문이다. 대표 지표는 `cluster_monitor_exporter_last_collection_timestamp_seconds` 와 `cluster_monitor_exporter_metrics_stale_after_seconds` 다.

수집/렌더링 구현은 `main.go`에서 확인한다.

관측 범위

- required mount의 source/target/filesystem/security option과 응답 시간
- storage host 및 peer 연결성, mount failure diagnosis
- 모든 thread를 포함한 host D-state process와 command별 count
- kernel hung-task 증가, NFS lease와 forensic 상태
- host `nvidia-smi` , Docker daemon
- 대상 image container의 running/SSH/GPU readiness
- NVML driver/library mismatch와 제한된 repair 결과
- 외부 connectivity heartbeat와 public inbound health
- NFS GSS readiness/canary/recovery state와 `rpc-gssd` journal

D-state와 명령 timeout

NFS `hard` mount에서 child process가 D-state에 들어가면 `SIGKILL` 에도 즉시 끝나지 않고 `cmd.Wait()` 가 무기한 block될 수 있다. exporter는 외부 명령마다 timeout을 두고 collection loop가 한 NFS probe 때문에 완전히 멈추지 않게 한다.

D-state는 발생 즉시 total과 command별 metric으로 기록하되 alert는 30분 연속 지속을 기다린다. `sshd` , `find` , `du` , `chown` , `chmod` , `codex` , `claude` , `python` , `python3` 은 0인 시계열도 유지하고, 다른 command는 실제 관측될 때만 동적으로 노출한다. hung-task alert는 과거 누적 line이 아니라 최근 5분 증가분을 사용한다.

제한된 container 복구

설정으로 허용된 경우에만 다음을 수행한다.

- stopped target container 시작

- container 내부 SSH service 시작
- host driver version 검증 뒤 `libnvidia-ml.so.1` symlink repair

임의 image/container, user data, host mount를 변경하지 않는다. 복구 시도와 결과는 별도 metric으로 남겨 “정상”과 “복구되어 정상”을 구분한다.

4. gpu-user-exporter

`nvidia-smi` 는 PID와 GPU 사용량만 제공하므로 실제 사용자와 container를 알기 위해 여러 정보를 join한다.

flowchart LR

```
SMI["nvidia-smi<br/>GPU/PID/memory/util"] --> JOIN["gpu-user-exporter"]
PROC["/proc/PID/cgroup<br/>container ID"] --> JOIN
DOCKER["Docker inspect<br/>state/device request"] --> JOIN
DB["UID DB cache<br/>owner/real name"] --> JOIN
JOIN --> METRIC["사용자-container-GPU별<br/>memory/util/process metric"]
```

주요 지표:

- `docker_gpu_user_memory_used_bytes`
- `docker_gpu_user_sm_utilization_percent`
- `docker_gpu_user_process_count`
- device 전체 utilization/memory
- ignored process count
- DB cache entry/refresh success
- exporter scrape success/duration

DB에 없는 process를 임의 사용자에게 귀속하지 않고 ignored count로 분리한다. 매 scrape마다 DB를 조회하지 않도록 활성 container owner를 cache하고 refresh 성공과 cache 크기를 metric으로 노출한다. 구현은 [gpu-user-exporter main.go](#)에 있다.

5. NFS GSS readiness, canary와 recovery

Readiness

부팅 시 NFS mount가 Kerberos host identity보다 먼저 실행되는 race를 막기 위해 다음 stage를 순서대로 확인한다.

```
host keytab -> machine kinit -> NFS service kvno
-> rpc_pipefs -> rpc-gssd -> (환경별 canary)
```

각 stage와 diagnosis는 state file에 기록되고 exporter가 metric으로 변환한다. 구현은 [readiness script](#)와 [NFS GSS collector](#)에 있다.

Canary

canary는 실제 user share가 아니라 전용 read-only export에서 service path를 확인한다. LAB은 canary를 사용하며 FARM은 전용 export가 준비되기 전까지 canary gate만 비활성화하고 나머지 readiness를 유지한다.

Guarded missing-mount recovery

exporter는 recovery state를 관측하고 loopback API로 worker를 queue할 수 있지만 실제 `mount` 는 별도 systemd service가 수행한다. worker gate는 다음과 같다.

1. 기대 source/filesystem/security로 이미 mount되어 있으면 즉시 healthy로 종료한다.
2. mount가 없을 때만 fstab 기대 entry를 확인한다.
3. NFS/RPC D-state가 있으면 새 mount 시도를 보류한다.
4. DNS와 storage RPC를 확인한다.
5. Kerberos readiness를 확인한다.
6. 환경에서 요구하는 canary를 확인한다.
7. retry/backoff/inhibit가 허용할 때만 `mount <target>` 을 실행한다.

healthy mount를 unmount/remount하지 않으며, mount timeout이나 관련 D-state가 생기면 현재 boot에서 반복 시도를 inhibit할 수 있다. 구현은 [mount recovery script](#)에 있다.

6. NFS forensics

NFS 장애는 재시작 후 증거가 사라질 수 있으므로 recovery와 독립된 passive forensics를 둔다.

- TCP/2049 packet의 크기 제한 ring
- 저용량 ftrace instance
- 5초 간격 kernel/NFS/D-state 상태 관측
- incident threshold에서 pre-incident ring을 local snapshot으로 보존
- PID와 process start time을 함께 사용해 PID 재사용으로 duration이 이어지지 않게 함

forensics는 mount/unmount, user share 읽기, NFS/GSS restart, recovery, reboot를 하지 않는다. exporter는 `/run/decs-nfs-forensics/status.json` 만 읽는다. `sec=krb5` 는 payload privacy를 제공하지 않으므로 capture와 incident directory는 root-only다.

구현은 [trace buffer](#), [watcher](#), [snapshot](#), [metric adapter](#)에서 확인한다.

7. Keytab health check

AD/NAS service keytab은 exporter의 자동 복구 대상이 아니다. 별도 user systemd timer가 profile별 checker를 실행한다.

```
AD KVN0 + required SPN
vs storage keytab KVN0/principal
+ 실제 service ticket 복호화
+ GSS service process/forbidden option
-> profile status JSON
```

exit code는 정상 0, 점검 오류 1, drift 2 다. keytab이나 credential 원문은 status에 기록하지 않는다. 코드는 [check-nfs-keytab.sh](#)에 있다.

8. Prometheus, Alertmanager와 Grafana

Cluster 분리

- FARM: `kube-prometheus-stack`, Prometheus/Grafana local PV, 공통 Grafana 제공
- LAB: LAB8 control plane의 독립 Prometheus, NodePort `30073`
- Grafana: LAB datasource UID `prometheus-lab` 으로 LAB dashboard 조회

retention, storage, target, Alertmanager routing과 control-plane failure domain이 다르므로 Prometheus release와 values를 분리한다. dashboard는 운영자가 한 UI에서 비교하므로 공통 디렉터리에 둔다.

Alert delivery

Prometheus rule은 Alertmanager로 전달되고, Alertmanager는 localhost relay sidecar에 native payload를 보낸다. relay가 내부 notify API contract로 변환하여 `/api/internal/slack/notify` 로 전달한다. Grafana password와 Slack webhook은 values가 아니라 Kubernetes Secret으로 관리한다.

canonical FARM rule과 routing은 `prometheus-farm-values.yaml`, relay 구현은 Alertmanager relay ConfigMap에서 확인한다.

9. 설정과 코드 위치

관심사	기준 파일
exporter 공통 기본값, FQDN mount/SPN, GSS/forensics option	<code>group_vars/exporters.yaml</code>
exporter build/install/systemd config	<code>deploy_exporters.yaml</code>
GSS readiness/canary/recovery 설치	<code>deploy_nfs_gss_health.yaml</code>
forensics 설치	<code>deploy_nfs_forensics.yaml</code>
FARM Prometheus/Alertmanager rules	<code>prometheus-farm-values.yaml</code>
LAB Prometheus values	<code>prometheus-lab-values.yaml</code>

관심사	기준 파일
GPU/NIC/storage dashboards	grafana/dashboards

10. 설계 변경 시 확인할 항목

- 새 metric의 이름, type, help, label cardinality와 실패 시 값
- command timeout과 NFS D-state에서 collection loop가 유지되는지
- alert **for** 시간이 일시적 부팅/배포 지연보다 충분히 긴지
- recovery의 enable flag, precondition, retry/backoff, inhibit와 audit metric
- user data를 읽지 않는 canary/probe 경로
- FARM/LAB datasource, job, label과 dashboard query의 일치
- credential, webhook, DB password가 Git/metric/label/log에 노출되지 않는지

Monitoring 운영

이 문서는 monitoring component를 배포하고 endpoint, metric, alert와 incident를 점검하는 방법을 설명한다.

1. 배포 순서

새 host 또는 NFS GSS monitoring을 처음 적용할 때는 다음 순서를 권장한다.

1. inventory와 host FQDN/SPN/mount source 확인
2. NFS GSS readiness/canary/recovery worker 설치
3. NFS forensics 설치
4. exporter 배포
5. Prometheus target/rule 반영
6. Grafana datasource/dashboard 확인
7. alert delivery test

운영 배포 entry point는 exporter 내부 개별 shell script가 아니라 **monitoring/ansible_playbook/** 이다.

2. Exporter 배포

```
cd /home/jy/server_manage/monitoring
```

두 exporter를 한 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=farm8
```

FARM 전체 또는 FARM/LAB 전체:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=FARM
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml
```

한 exporter만 전체 재배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_gpu_user_exporter_all.yml
```

target에는 다음 systemd service와 endpoint가 생긴다.

```
cluster-monitor-exporter.service -> :30074/metrics, :30074/healthz, :N89/healthz  
gpu-user-exporter.service        -> :30072/metrics, :30072/-/healthy
```

배포 기준과 요구 조건은 [Ansible README](#), 실제 task는 [deploy_exporters.yml](#)에서 확인한다.

3. NFS GSS와 forensics 배포

NFS GSS readiness/canary/recovery worker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \  
-e nfs_gss_health_hosts=lab8
```

NFS forensics:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

중요한 systemd unit과 state:

```
decs-kerberos-nfs-ready.service  
decs-nfs-gss-canary-probe.service/.timer  
decs-nfs-gss-mount-recovery.service/.timer  
/run/decs-nfs-gss/ready.state  
/var/lib/decs-nfs-gss/canary.state  
/var/lib/decs-nfs-gss/recovery.state  
  
decs-nfs-forensics-buffer.service  
decs-nfs-forensics-watch.service/.timer  
decs-nfs-forensics-snapshot.service  
/run/decs-nfs-forensics/status.json  
/var/lib/decs-nfs-forensics/
```

FARM은 전용 read-only canary export가 준비될 때까지 canary만 비활성화하며 keytab/kinit/kvno/rpc-gssd readiness와 guarded recovery는 유지한다. LAB canary는 `sec=krb5` 전용 read-only export를 사용한다.

4. Keytab checker 배포

FARM 읽기 전용 checker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml
```

LAB을 상시 점검 대상으로 전환한 뒤에만 두 profile을 활성화한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml \  
-e '{"keytab_check_profiles":["farm","lab"]}'
```

```
systemctl --user list-timers 'check-nfs-keytab@*.timer'  
systemctl --user status check-nfs-keytab@farm.service --no-pager
```

checker는 drift를 고치지 않는다. KVNO history, repair와 rotation은 [Kerberos/NFS 운영](#)을 따른다.

5. Prometheus/Alertmanager 배포

먼저 server-side render와 cluster precondition만 검증한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml
```

변경 적용:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml \  
-e prometheus_dry_run=false
```

한 환경만 배포하려면 `prometheus_farm_enabled=false` 또는 `prometheus_lab_enabled=false` 를 사용한다. playbook은 kubeconfig 대상 cluster, node Ready/DiskPressure, 필수 Secret 이름을 확인한 후 Helm을 실행한다.

FARM에서 필요한 Secret:

- `monitoring-grafana-admin`: `admin-user`, `admin-password`
- `cluster-monitor-slack-webhook-farm`: `url`

값은 Git values에 넣지 않는다. Alertmanager는 Slack webhook으로 직접 보내지 않고 localhost relay를 거쳐 internal notify API로 보낸다.

6. 배포 후 endpoint 확인

target host에서:

```
systemctl status cluster-monitor-exporter gpu-user-exporter --no-pager  
journalctl -u cluster-monitor-exporter -n 100 --no-pager  
journalctl -u gpu-user-exporter -n 100 --no-pager  
  
curl -fsS http://127.0.0.1:30074/healthz  
curl -fsS http://127.0.0.1:30074/metrics | head  
curl -fsS http://127.0.0.1:30072/-/healthy  
curl -fsS http://127.0.0.1:30072/metrics | head
```

public health port는 서버 번호 `N` 에 대해 `9000 + N*100 - 11` 이다. 예를 들어 FARM8은 `9789/healthz` 다. 이 endpoint는 exporter process/collection freshness와 외부 NAT 도달 경로를 확인하는 용도이지, 모든 dependency의 readiness를 대신하지 않는다.

NFS GSS host-only API:

```
curl -fsS http://127.0.0.1:30074/nfs-gss/ready
curl -fsS http://127.0.0.1:30074/nfs-gss/health
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/probe
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/recover
```

probe 와 **recover** 는 loopback 요청만 허용하며 worker를 queue한 뒤 즉시 응답한다. **recover** 를 반복 호출하기 전에 recovery state와 inhibit reason을 확인한다.

7. 무엇을 어디에서 보는가

관측 대상	대표 metric/alert	구현/설정 링크
exporter process와 freshness	<code>up</code> , <code>cluster_monitor_exporter_last_collection_timestamp_seconds</code> , <code>ClusterMonitorExporter*</code>	collector, rules
required mount와 latency	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , <code>..._probe_seconds</code>	mount collector, storage dashboards
Kerberos NFS readiness	<code>cluster_monitor_nfs_gss_ready</code> , <code>ClusterMonitorNFSGSSReadinessFailed</code>	readiness, GSS collector
canary/recovery	<code>cluster_monitor_nfs_gss_canary_*</code> , <code>..._recovery_*</code>	canary, recovery
D-state, lease, hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code> , <code>ClusterMonitorNFS*</code>	forensics adapter, forensics scripts
host GPU/Docker/container	<code>cluster_monitor_host_gpu_up</code> , <code>...docker_daemon_up</code> , <code>...container_*</code>	cluster collector
사용자별 GPU	<code>docker_gpu_user_memory_used_bytes</code> , <code>...sm_utilization_percent</code> , <code>...process_count</code>	gpu-user-exporter, GPU dashboard
외부 연결	<code>cluster_monitor_external_connectivity_*</code> , public health	exporter, Apps Script
AD/NAS keytab	checker JSON/exit code	keytab checker

canonical alert rule은 [prometheus-farm-values.yaml](#)이다. exporter 내부

`prometheus/cluster_monitor_alerts.yml` 은 metric 이름을 보여 주는 reference이며 실제 FARM release와 값이 다를 수 있다.

8. Alert별 진단 순서

Exporter down 또는 stale

1. Prometheus target에서 connection error와 scrape timestamp를 구분한다.

2. target systemd status/journal을 확인한다.
3. `:30074/healthz` 와 `/metrics` 를 각각 확인한다.
4. process는 살아 있지만 stale이면 마지막 실행 command와 D-state를 확인한다.
5. 변경된 env/config와 binary 배포 시간을 확인한 뒤 exporter만 재배포한다.

Required mount down/slow/unresponsive

1. `findmnt` 로 mount 존재, FQDN source, filesystem과 `sec` 를 확인한다.
2. mount probe와 storage ping/peer diagnosis label을 확인한다.
3. GSS readiness에서 처음 실패한 stage를 확인한다.
4. D-state, lease expiry, hung-task와 local forensic snapshot을 확인한다.
5. mount가 없을 때만 recovery state를 확인한다.
6. mount가 이미 있거나 D-state caller가 있으면 강제 remount를 반복하지 않는다.

Kerberos source는 IP가 아니라 FQDN이어야 한다. 자세한 기준은 [Kerberos/NFS 운영의 mount 절](#)을 따른다.

GSS readiness/canary/recovery

```
systemctl status decs-kerberos-nfs-ready.service --no-pager
systemctl status decs-nfs-gss-canary-probe.timer --no-pager
systemctl status decs-nfs-gss-mount-recovery.timer --no-pager
journalctl -u decs-kerberos-nfs-ready.service -n 100 --no-pager
cat /run/decs-nfs-gss/ready.state
cat /var/lib/decs-nfs-gss/canary.state
cat /var/lib/decs-nfs-gss/recovery.state
```

- keytab/kinit stage: host machine credential 확인
- kvno stage: DNS/FQDN/SPN/KDC 확인
- rpc-gssd stage: service와 잘못된 `-n` override 확인
- canary stage: 전용 export와 user share를 혼동하지 않았는지 확인
- inhibited recovery: D-state 또는 mount timeout 증거를 보존하고 원인을 처리

GPU 사용자 mapping 이상

1. `nvidia-smi` 와 exporter scrape success를 확인한다.
2. ignored process count가 증가했는지 확인한다.
3. PID의 cgroup container ID와 Docker inspect를 비교한다.
4. DB cache refresh success와 cache age/entry를 확인한다.
5. DB record와 실제 running container가 어긋났다면 user-lifecycle 소유 절차로 고친다.

Container SSH/GPU alert

1. container가 running인지 먼저 확인한다.
2. exporter가 start/SSH/NVML recovery를 시도했는지 metric과 journal을 확인한다.
3. container image가 configured regex 대상인지 확인한다.
4. NVML mismatch가 아니라 application/GPU allocation 문제면 자동 symlink repair를 반복하지 않는다.

9. Grafana 점검

- LAB dashboard datasource UID가 `prometheus-lab` 인지 확인한다.
- dashboard query의 `cluster`, `server_id`, `job` label이 scrape config와 일치하는지 확인한다.
- storage dashboard에서 mount probe, responsive, D-state, blocked process, hung-task를 같은 시간축으로 비교한다.
- GPU dashboard에서 DB-known 사용자 metric과 device 전체 metric을 함께 본다.
- 새 server 추가 시 dashboard에 hard-coded server/GPU panel 누락이 없는지 확인한다.

dashboard 원본:

- [FARM storage latency](#)
- [LAB storage latency](#)
- [GPU usage](#)
- [Network traffic](#)

10. 테스트

```
cd /home/jy/server_manage/monitoring

(cd prometheus/exporters/cluster-monitor-exporter && go test ./...)
(cd prometheus/exporters/gpu-user-exporter && go test ./...)
bash nfs-forensics/tests/test_watch.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
python3 prometheus/config/tests/test_slack_notify_relay.py
```

변경 위험에 맞게 최소한 다음을 함께 검증한다.

- metric 이름/type/help와 0/failure path
- recovery가 healthy mount를 건드리지 않는지
- D-state에서 mount attempt가 차단되지만 existing mount는 healthy로 처리되는지
- Alertmanager relay가 secret을 출력하지 않고 internal API payload로 변환하는지
- FARM/LAB dashboard datasource가 바뀌지 않았는지

11. 새 서버 추가 체크리스트

1. `ansible_playbook/inventory.ini` 에 `farmN/ LabN` 형식으로 추가
2. `group_vars/exporters.yml` 의 mount source, SPN과 환경 기본값 확인
3. host share target와 public `N89` port 계산 확인
4. GSS readiness/forensics/exporter 배포
5. Prometheus `:30070` , `:30072` , `:30074` scrape target 추가
6. 방화벽/NAT public health 확인
7. Grafana dashboard server/GPU panel 확인
8. alert rule label과 Alertmanager route 확인
9. endpoint와 실제 alert delivery 검증

12. 운영 문서에 계속 추가할 내용

설계와 운영을 분리한 뒤 다음 정보를 운영 문서에 누적하면 좋다.

- 서비스별 owner, endpoint, systemd/Kubernetes resource와 dependency
- metric별 정상 범위, alert `for` 와 runbook link
- 최근 배포 version/commit과 마지막 검증 시각
- FARM/LAB retention, PV 용량과 capacity threshold
- alert silence 승인/만료 기준과 false-positive 기록
- 자동 복구별 시도 횟수, inhibit 해제와 rollback 절차
- incident snapshot 보존 기간, 접근 권한과 개인정보 취급
- 새 server/metric/dashboard 추가 checklist

설계 문서에는 component 관계와 안전 경계를 유지하고, host별 현재 값과 일회성 incident 결과는 canonical config/runbook 또는 별도 evidence에 기록한다.